



Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas

HubArena

Relatório Técnico

Aluno: Rael Kiluanji de Jesus Cassimiro

Belo Horizonte – MG
2026

1. Introdução

O HubArena é uma aplicação distribuída para gerenciamento de arenas esportivas e reservas. Durante as quatro sprints do projeto foram implementados um backend REST em Flask, persistência em PostgreSQL, comunicação assíncrona por RabbitMQ e um aplicativo Flutter unificado para clientes e prestadores. O objetivo deste relatório é descrever a arquitetura implementada, justificar as principais decisões de projeto, apresentar as dificuldades encontradas e refletir sobre os padrões de engenharia de software estudados ao longo da disciplina.

2. Arquitetura Implementada

A solução é composta por cinco componentes principais: aplicativo Flutter, backend Flask, PostgreSQL, RabbitMQ e um consumidor de eventos. O aplicativo comunica-se com o backend por meio de uma API REST. O backend centraliza as regras de negócio, persiste os dados no PostgreSQL e publica eventos relacionados às reservas no RabbitMQ. O consumidor processa esses eventos de forma assíncrona, reduzindo o acoplamento entre os componentes.

A organização do backend segue uma arquitetura em camadas composta por rotas, controladores, serviços, repositórios e modelos. Essa separação permite isolar a lógica de negócio da camada de persistência e da infraestrutura, facilitando manutenção, testes e evolução do sistema.

3. Decisões de Design

Optou-se pelo Flask devido à simplicidade para construção de APIs REST e integração com SQLAlchemy. O PostgreSQL foi escolhido por sua robustez e suporte a transações. O RabbitMQ foi adotado como middleware orientado a mensagens para permitir comunicação assíncrona entre os componentes. O Flutter foi utilizado para desenvolver uma aplicação móvel única, contemplando os perfis de cliente e prestador por meio de interfaces distintas.

Também foi adotada geração automática de imagens para arenas e quadras quando o usuário não informa uma URL, permitindo uma melhor experiência de uso sem depender de upload de arquivos.

4. Dificuldades Encontradas e Soluções

- **Integração Flutter e backend local:** Utilização do endereço 10.0.2.2 no emulador Android para acessar o servidor Flask.
- **Sincronização das notificações:** Ajuste do gerenciamento de estado para evitar inconsistências na interface.
- **Comunicação assíncrona:** Implementação de produtor e consumidor RabbitMQ para desacoplar o processamento de eventos.
- **Modelagem do domínio:** Separação entre usuários, arenas, quadras e reservas para facilitar a evolução do sistema.

5. Reflexão sobre os Padrões Estudados

5.1 REST

A API foi estruturada segundo princípios REST, com recursos bem definidos, utilização dos métodos HTTP apropriados e comunicação stateless baseada em JSON.

5.2 Clean Architecture

A separação entre controllers, services, repositories e models segue os princípios da Clean Architecture, mantendo as regras de negócio independentes da infraestrutura.

5.3 MOM

O RabbitMQ foi empregado como Message-Oriented Middleware para desacoplar a publicação e o consumo de eventos, permitindo maior escalabilidade e flexibilidade.

5.4 Event-Driven Architecture (EDA)

O sistema publica eventos como criação, aceite, cancelamento e finalização de reservas. Os consumidores podem reagir a esses eventos independentemente do fluxo principal da aplicação.

6. Conclusão

O HubArena atingiu os objetivos propostos pela disciplina, integrando comunicação síncrona e assíncrona em uma arquitetura distribuída. A organização em camadas, o uso de REST e RabbitMQ e a integração entre backend e aplicativo Flutter resultaram em uma solução modular e preparada para evoluções futuras, como autenticação JWT, notificações push e implantação em nuvem.

Referências Bibliográficas

- BAILEY, Thomas. Flutter for Beginners. 3rd ed. Packt, 2023.
- COULOURIS, George et al. Distributed Systems: Concepts and Design. 5th ed. Addison-Wesley, 2011.
- HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns. Addison-Wesley, 2003.
- MARTIN, Robert C. Arquitetura Limpa. Alta Books, 2019.
- RICHARDSON, Chris. Microservices Patterns. Manning, 2018.